Мониторинг компьютерных атак

Практика 6. Аудит репозитория

Semgrep · Syft и Grype · Gitleaks · план устранения · мониторинг

Совместный вариант А — каталог библиотеки на Python

Ход практики

Шаг	Что делаем	Мин
0	Запускаем контейнер, ставим инструменты, клонируем репозиторий	15
1	Анализ кода — Semgrep	25
2	Состав ПО — Syft и Grype, второй SBOM, достижимость	25
3	Секреты — Gitleaks	10
4	План устранения по BOD 26-04, исключение, CI/CD	25
5	Мониторинговый мост	15

Вместе — вариант A. Затем самостоятельно — вариант B или C.

Что проверяем: каталог библиотеки

Небольшой веб-сервис на Python (Flask), 3 файла кода:

- `app.py` — поиск книг, вход библиотекаря, удаление книги, импорт списка книг из YAML, проверка связи с партнёром, обложки
- `db.py` — подключение к базе SQLite
- `notify.py` — уведомления в Slack
- `requirements.txt` — библиотеки, которые нужны сервису

Четыре коммита истории. Репозиторий — файл `library.bundle`: клонируется как обычный репозиторий.

Все «секреты» в репозитории сгенерированы и нигде не действуют.

Шаг 0. Контейнер и инструменты

0.1. Запуск контейнера — на своём компьютере

Нужен Docker: Docker Desktop (Windows, macOS) или Docker Engine (Linux).

```
docker version
mkdir -p lab06/work && cd lab06
curl -sSfL -o work/library.bundle \
  https://github.com/mriddi/practic_6/raw/main/repos/library.bundle
docker run -it --name audit -v "$PWD/work:/work" -w /work python:3.11-bookworm bash
```

- варианты B и C: `shop.bundle`, `clinic.bundle` — тот же адрес, другое имя файла
- Windows PowerShell: `-v "${PWD}\work:/work"`
- вернуться после выхода: `docker start -ai audit`
- второй терминал: `docker exec -it audit bash`
- результаты остаются на компьютере в `lab06/work/`

0.2. Проверка среды и Semgrep — в контейнере

Среда

```
cat /etc/debian_version
uname -m
python3 --version; pip --version; git --version
```

Semgrep — точная версия, отдельное окружение

```
python3 -m venv /opt/semgrep
/opt/semgrep/bin/pip install -q "semgrep==1.136.0"
ln -s /opt/semgrep/bin/semgrep /usr/local/bin/semgrep
semgrep --version
```

0.3. Syft и Grype: версия и контрольная сумма

Архитектура и каталог загрузки

```
ARCH=$(dpkg --print-architecture); echo $ARCH
mkdir -p /tmp/dl && cd /tmp/dl
```

Syft и Grype: архив, файл сумм, проверка, установка

```
V=1.54.1; U=https://github.com/anchore/syft/releases/download/v$V
curl -sSfLO $U/syft_${V}_linux_${ARCH}.tar.gz
curl -sSfLO $U/syft_${V}_checksums.txt
sha256sum --check --ignore-missing syft_${V}_checksums.txt
tar xzf syft_${V}_linux_${ARCH}.tar.gz syft && install syft /usr/local/bin/
V=0.120.1; U=https://github.com/anchore/grype/releases/download/v$V
curl -sSfLO $U/grype_${V}_linux_${ARCH}.tar.gz
curl -sSfLO $U/grype_${V}_checksums.txt
sha256sum --check --ignore-missing grype_${V}_checksums.txt
tar xzf grype_${V}_linux_${ARCH}.tar.gz grype && install grype /usr/local/bin/
```

0.4. Gitleaks и проверка установки

Gitleaks: у архива для amd64 суффикс x64

```
V=8.30.1; U=https://github.com/gitleaks/gitleaks/releases/download/v$V
GA=$( [ $ARCH = amd64 ] && echo x64 || echo $ARCH )
curl -sSfLO $U/gitleaks_${V}_linux_${GA}.tar.gz
curl -sSfLO $U/gitleaks_${V}_checksums.txt
sha256sum --check --ignore-missing gitleaks_${V}_checksums.txt
tar xzf gitleaks_${V}_linux_${GA}.tar.gz gitleaks && install gitleaks /
usr/local/bin/
```

Проверка

```
cd /work
syft version | head -2
grype version | head -2
gitleaks version
```

0.5. Настройка и целевой репозиторий

Отключить проверку обновлений

```
echo 'check-for-app-update: false' > ~/.syft.yaml
echo 'check-for-app-update: false' > ~/.grype.yaml
```

Клон из файла bundle, история коммитов

```
ls -l *.bundle
git clone -q library.bundle target-repo
git -C target-repo log --format="%h %ad %s" --date=short
mkdir -p results
ls -a target-repo
```

Шаг 1. Анализ кода — Semgrep

1.1. Запуск Semgrep

Два набора правил, без отправки метрик

```
semgrep scan --config p/default --metrics=off --json-
output=results/semgrep.json --text-output=results/semgrep.txt
target-repo/
semgrep scan --config p/owasp-top-ten --metrics=off --json-
output=results/semgrep-owasp.json target-repo/
```

1.2. Уровни, CWE, правила, повторы — по JSON-отчёту

```
grep -o '"severity":"[A-Z]*"' results/semgrep.json | sort |
    uniq -c
grep -o '"CWE-[0-9]*' results/semgrep.json | sort | uniq -c | sort
-rn | head -5
grep -o '"check_id":"[^"]*"' results/semgrep.json | sort | uniq -c
| sort -rn | head -5
grep -o '"check_id":"[^"]*","path":"[^"]*","start":{"line":[0-9]*'
results/semgrep.json |
    sed 's/"check_id":"[^"]*",//' | sort | uniq -c | awk '$1 > 1'
```

1.3. Находки ERROR с кодом

```
semgrep scan --config p/default --metrics=off --severity ERROR
target-repo/
```

1.4. Код вокруг находок

```
sed -n '15,21p' target-repo/app.py
sed -n '45,50p' target-repo/app.py
sed -n '56,62p' target-repo/app.py
sed -n '65,69p' target-repo/app.py
```

1.5. Что Semgrep не нашёл

```
sed -n '37,44p' target-repo/app.py
grep -n "secret_key" target-repo/app.py
```

Шаг 2. Состав ПО — Syft и Grype

2.1. SBOM по репозиторию: из чего он построен

```
cat target-repo/requirements*.txt
syft dir:target-repo -o table -o cyclonedx-json=results/sbom.json
2>/dev/null > results/sbom.txt
cat results/sbom.txt
grep -o '"syft:location:0:path","value":"[^"]*"' results/sbom.json
| sort | uniq -c
```

2.2. Grype: база, таблица по RISK, уровни, KEV

```
grype db update
grype db status
grype sbom:results/sbom.json --by-cve -o table -o json=results/
grype.json > results/grype.txt
head -11 results/grype.txt
tail -n +2 results/grype.txt | wc -l
grep -oE ' (Critical|High|Medium|Low|Negligible|Unknown) ' results/
grype.txt |
    sort | uniq -c | sort -rn
awk 'NR > 1 {print $1, $2}' results/grype.txt | sort |
    uniq -c | sort -rn
grep kev results/grype.txt
```

2.3. Второй SBOM: что устанавливается на самом деле

Установить зависимости в отдельный каталог, ничего не запуская

```
pip install -q --disable-pip-version-check --root-user-
action=ignore --target libs -r target-repo/requirements.txt
ls libs
```

SBOM по установленным пакетам и разница двух SBOM

```
syft dir:libs -o table -o cyclonedx-json=results/sbom-
installed.json 2>/dev/null > results/sbom-installed.txt
cat results/sbom-installed.txt
diff <(awk 'NR > 1 {print $1,
    $2}' results/sbom.txt) <(awk 'NR > 1 {print $1,
    $2}' results/sbom-installed.txt)
```

2.4. Второй SBOM: уязвимости

```
grype sbom:results/sbom-installed.json --by-cve -o table -o
json=results/grype-installed.json > results/grype-installed.txt
tail -n +2 results/grype-installed.txt | wc -l
awk 'NR > 1 {print $1, $2}' results/grype-installed.txt |
    sort | uniq -c | sort -rn
```

2.5. Через что пришёл пакет

```
grep -l "^Requires-Dist: urllib3" libs/*.dist-info/METADATA
grep "^Requires-Dist: urllib3" libs/requests-2.28.0.dist-info/
METADATA
grep "^Requires-Dist" libs/[Ff]lask-2.2.2.dist-info/METADATA
cd /tmp && PYTHONPATH=/work/libs python3 -c "import flask" 2>&1 |
    tail -1
```

2.6. Достижимость: используется ли библиотека

```
grep -rn "^import\|^from" target-repo --include=*.py
grep -rn "PIL\|Image" target-repo --include=*.py
grep -rn "requests\.\|yaml\." target-repo --include=*.py
```

2.7. CVSS и оценки CISA — в NVD

Для пяти первых CVE из таблицы Grype откройте

```
https://nvd.nist.gov/vuln/detail/CVE-2023-4863
```

- **CVSS 3.x** — блок оценок NVD и поставщиков
- **CISA ADP Vulnrichment** → SSVC: Exploitation, **Automatable**, **Technical Impact** (total = полный контроль)
- нет оценки CISA — оцените сами и обоснуйте
- KEV: пометка `(kev)` в таблице Grype или поиск на cisa.gov/known-exploited-vulnerabilities-catalog

Шаг 3. Секреты — Gitleaks

3.1. Текущие файлы и история

```
gitleaks dir target-repo -v --redact --report-format json --report-
path results/gitleaks-dir.json > results/gitleaks-dir.txt 2>&1
tail -1 results/gitleaks-dir.txt
grep -E '^(RuleID|File|Line):' results/gitleaks-dir.txt
gitleaks git target-repo -v --redact --report-format json --report-
path results/gitleaks.json > results/gitleaks-git.txt 2>&1
tail -1 results/gitleaks-git.txt
grep -E '^(RuleID|File|Commit|Date):' results/gitleaks-git.txt
grep '^File' results/gitleaks-git.txt | awk '{print $2}' |
    sort -u | while read f; do [ -e "target-repo/$f" ] ||
    echo "$f"; done
```

3.2. Что за секреты и где они

```
git -C target-repo log --oneline --stat -- .env
git -C target-repo show 4b470b8:.env |
    sed -E 's/(KEY[A-Z_]*=.{4}).*/\1…/'
grep -n "SLACK_URL =" target-repo/notify.py | cut -c1-50
```

Шаги 4–5. План и мониторинг

4. План устранения — вопросы к каждой находке

1. Доступна из интернета? **(считаем — да)** · В KEV? · Автоматизируема? · Полный или частичный контроль?
2. Срок по таблице BOD 26-04; секреты — немедленно, вне таблицы
3. Достижимость: используется ли библиотека? Если нет — запись VEX «не затронут»
4. План: не меньше 10 строк — действие, владелец, **проверка**
5. Сравнить с порядком по CVSS: кто поднялся, кто опустился и почему
6. Одно исключение: причина, компенсирующая мера, владелец, срок
7. CI/CD: где запускается каждый инструмент, что блокирует, что предупреждает

5. Мониторинговый мост

Для трёх находок, которые закрываются не сразу:

- **что ищем** — конкретный адрес и параметр запроса, процесс, соединение
- **в каком журнале** — и пишется ли туда нужное поле сейчас
- **попытка или успех** — какое второе событие подтверждает атаку
- **ложные срабатывания**

Для секрета: какой журнал, какой фильтр, за какой период — и хватит ли срока хранения.

Две строки каталога событий цепочки поставок.

Что сдаём: содержимое results/

```
ls -l results/
```

Самостоятельно: вариант B или C

	B — интернет-магазин (Flask)	**C — запись к врачу (Django)**
Контейнер	Новый каталог, в `work/` — файл `shop.bundle`; шаг 0 целиком	Новый каталог, в `work/` — файл `clinic.bundle`; шаг 0 целиком
Клон	`git clone -q shop.bundle target-repo`	`git clone -q clinic.bundle target-repo`
Шаги 1–3	Те же команды, что в варианте A: 1.1–1.3, 2.1–2.6, 3.1 и поиск в NVD (2.7). Код вокруг находок, путь пакета и детали секретов (1.4, 2.5, 3.2) — смотрите сами по своему выводу	
Подумать	Что Syft взял из `requirements-dev.txt` и нужно ли это в SBOM? Что из найденного Gitleaks — тестовое?	Почему в SBOM по репозиторию нет `requests` и `psycopg2-binary`? Какие секреты в коде не нашёл ни один инструмент?
Сдать	`results/*.json`, `results/plan.md` (разбор, план, исключение, CI/CD), `results/monitoring.md` (правила, проверка секрета, каталог) — по README практики	